

Team 2 SC5006 Project

ALBINOWSKI WIKTOR MARCIN (N2500433A)
YAO MUHENG (YAOM0008)
MANOHAR RAJA AGATHIYAN (AGATHYA001)
OKUMURA MISAKI (MISAKI001)

23rd of September 2026

1. Introduction

Object counting in open-world scenes requires a model to locate and count instances described at inference time, including categories that are not fixed by the evaluation interface. This project studies CountGD, which accepts a text query, one or more visual exemplar boxes, or both. A text query specifies the intended category in ordinary language. An exemplar supplies image-specific appearance information, which is useful when a category label is broad or when the target is better indicated by example. Using both inputs permits a direct comparison of text-only, exemplar-only, and multimodal counting under the same model and image.

Five connected components are examined: the image encoder, text encoder, ROIAlign exemplar extraction, query-prompt similarity matrix, and Top-K proposal selection. Their implementations are then exercised through live inference on `women.jpg` and `car.jpg`. The integrated experiment checks the shapes and masks produced by the components and compares localizations and counts across four baseline prompt modes. A car modality comparison isolates the effect of text and exemplar input. Prompt sensitivity is tested separately by changing "women" to "man" and "car" to "yellow car" while keeping the image fixed. Baselines use the standard maximum-token threshold $\tau = 0.23$, whereas the prompt comparison uses a meaningful-token confidence product with $\tau p = 0.50$. The provided counts of five women and sixteen cars are reference targets; measured results come only from live model outputs.

2. Related Work

Class-agnostic counting. Early counting methods were trained for a single category, such as people, cars or cells, and had to be retrained for every new category. Ranjan et al. [1] reframed counting as a few-shot problem: given a handful of exemplar boxes drawn on the image, the model predicts a density map covering every instance of that object. They also released FSC-147, a benchmark of 147 categories and over 6,000 images in which every image comes with three exemplar boxes, and whose training and test categories do not overlap, so models must count categories they have never seen. Later exemplar-based methods such as CounTR [2] improved accuracy with transformer architectures. However, all of them still need a user to draw exemplars at inference time.

Text-specified counting. To remove this manual step, later work specifies the target object in words. CLIP-Count [3] adapts the image-text alignment learned by CLIP [4] to dense image features. This allows a text prompt alone yields a density map. CounTX [5] trains a single-stage counter on top of joint image-text features and extends FSC-147 with richer class descriptions. Both methods regress a density map and sum it to obtain the count. Yet still, neither of all localises individual objects.

Open-vocabulary detection and grounding. A separate line of work detects objects named by free text. DETR [6] treats detection as predicting a set of objects from a fixed number of learned queries, and Deformable DETR [7] makes this practical on multi-scale feature maps by letting each query attend to only a few sampled locations. Grounding DINO [8] builds on this design for open-set detection. It fuses

image features from a Swin Transformer [9] with text features from BERT [10] and scores every query against every text token. This lets the detector be able to be prompted with any phrase.

CountGD. CountGD [11] turns Grounding DINO into a counter. Exemplar features are pooled from the image feature map with ROIAAlign [12] and inserted alongside the text tokens. The object can be specified by text, by exemplars, or by both together. Counting becomes detection. Each query is scored against the prompt tokens and those above a confidence threshold are counted. CountGD reports clear gains over prior methods on several counting benchmarks and its authors show that extra words such as “left” or “right” can restrict the count to a subset of the objects. We reproduce CountGD component by component and test how far this subset selection extends to colour attributes.

3. Method

3.1 Pipeline Overview

CountGD can use an image, a text prompt, exemplar boxes, or a combination of them as input. The image is first passed through an encoder to extract features at different scales. If a text prompt is provided, BERT is used to encode it. For exemplar boxes, ROIAAlign extracts features from the selected regions. In this way, the model gets both the visual information from the image and some guidance about the objects that need to be counted.

The whole model combines the image and prompt features and uses similarity scores to rank candidate locations. Then the Top-K selects candidates for the decoder, which predicts their boxes and token scores. We keep those predictions that pass the lowest confidence threshold and then count how many remain.

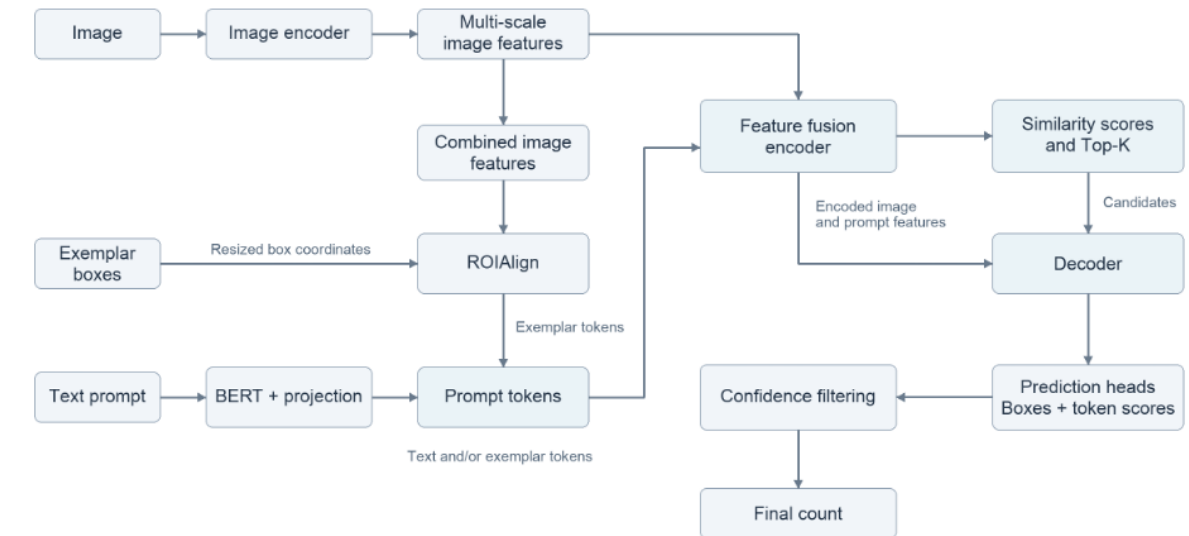


Figure 1: Simplified CountGD pipeline.

Algorithm 1 Exemplar tokens, similarity matrix and Top- K proposal selection

Require: combined image features $F \in \mathbb{R}^{B \times D \times H \times W}$ (stride 8); text tokens $P_{\text{text}} \in \mathbb{R}^{B \times T_t \times D}$ with validity mask m_{text} ; exemplar boxes $\{b_{b,e}\}$ in resized-image pixels (x_1, y_1, x_2, y_2) ; number of proposals K

Ensure: exemplar tokens E , similarity matrix S , top- K scores, proposal IDs and winning prompt IDs

1. ROIAlign exemplar tokens

1: **for** each image b and exemplar e **do**

2: $\tilde{b}_{b,e} \leftarrow b_{b,e} \cdot \frac{1}{8}$

▷ pixel box $\rightarrow$ feature-map box

3: $E_{b,e} \leftarrow \text{ROIALIGN}(F_b, \tilde{b}_{b,e}, \text{output} = 1 \times 1, \text{aligned})$

▷ bilinear samples, averaged; $E_{b,e} \in \mathbb{R}^D$

4: **end for**

2. Similarity matrix

5: $Q \leftarrow \text{FLATTEN}(F)$

▷ $Q \in \mathbb{R}^{B \times N \times D}$, $N = HW$

6: $P \leftarrow [P_{\text{text}}; E]$, $m \leftarrow [m_{\text{text}}; \mathbf{1}]$

▷ $P \in \mathbb{R}^{B \times T \times D}$

7: $S \leftarrow QP^\top$

▷ $S_{b,n,t} = \mathbf{q}_{b,n} \cdot \mathbf{p}_{b,t}$

8: $S_{b,n,t} \leftarrow -\infty$ **where** $m_{b,t} = 0$

▷ padding can never win

3. Top- K proposal selection

9: $s_{b,n} \leftarrow \max_t S_{b,n,t}$, $w_{b,n} \leftarrow \arg \max_t S_{b,n,t}$

▷ strongest prompt per position

10: $(\text{scores}_b, \text{ids}_b) \leftarrow \text{TOPK}(s_b, K)$

▷ over the N positions

11: $\text{prompt_ids}_b \leftarrow w_b[\text{ids}_b]$

12: **return** E , S , scores, ids, prompt_ids

3.2 Image Encoder

The implemented `encode_image` function accepts either a tensor with shape (B, 3, H, W) or a list of (3, H, W) tensors. It converts the input to CountGD’s NestedTensor representation, which stores the padded image batch and a mask that distinguishes image pixels from padding, and moves both to the active model device. Under `torch.inference_mode()`, `model.backbone` applies the Swin image encoder and returns multi-scale feature NestedTensors together with positional encodings. `model.combine_features` then resizes and fuses the backbone outputs to form the 1/8-scale feature map used for exemplar extraction. The function returns the original feature list, positional encodings, and combined map so that the intermediate stages remain available for inspection.

For the transformed car image, the input tensor was (3, 800, 1066). The three backbone features had shapes (1, 256, 100, 134), (1, 512, 50, 67), and (1, 1024, 25, 34), with masks of (1, 100, 134), (1, 50, 67), and (1, 25, 34). Each positional encoding used 256 channels at the corresponding spatial resolution. Feature fusion produced a combined tensor of (1, 256, 100, 134). Its height is exactly one eighth of the transformed image height, while the width is rounded to 134 grid cells. Averaging this tensor over its 256 channels gave a spatial activation map. The map confirms that the encoder produced spatially varying responses, but it does not measure localization or counting accuracy.

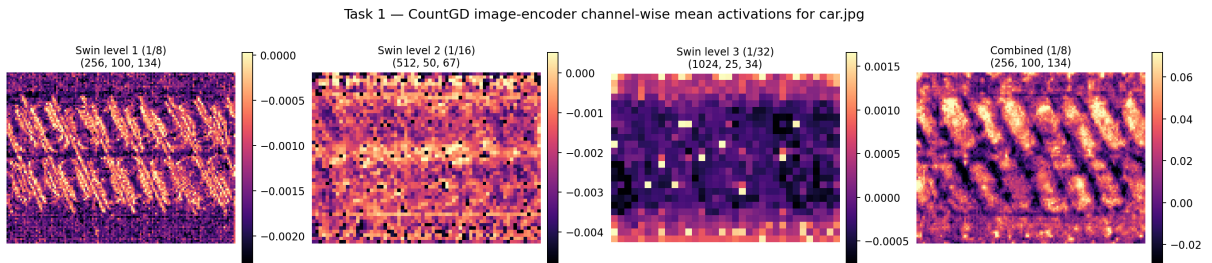


Figure 2: Multi-Scale Image Encoder Activations. Prompt token heatmaps

3.3 Text Encoder

The query is formatted as “*query* .” and tokenized with BERT’s WordPiece tokenizer [10], which adds [CLS] and [SEP]; for example, “car .” becomes four tokens. Captions in a batch are padded to the longest length of a caption in a batch, and a validity mask (`text_token_mask`) marks the padded columns. This matters because padded columns carry no meaning: if they were scored, a proposal could be counted for matching nothing. The mask is therefore used twice. Inside the feature enhancer, it is inverted so that no token attends to padding, and before similarity scoring the padded columns are filled with $-\infty$. This does not let the padded columns to be a proposal’s strongest prompt. Following Grounding DINO [8], a special-token attention mask restricts self-attention to tokens within the same phrase, delimited by “.”, and position IDs are assigned per phrase.

Figure 3 illustrates both masks for a batch of three captions of unequal length: “car .”, “yellow car .” and “red car . white car .”. In the self-attention panels, a white cell means that a row token attends to a column token. Each phrase forms its own square block. “red car . white car .” the two phrases appear as two separate blocks and neither can attend to the other, while [CLS] and [SEP] only attend to themselves. The right panel shows the validity mask: all three captions are padded to the length of the longest (eight tokens). “car .” has four real tokens followed by four [PAD] columns, and “yellow car .” has three shown in red.

Sequences longer than `max_text_len` = 256 are truncated, which is also why each forward pass returns 256 confidence columns, of which only the first few are real. BERT’s 768-dimensional outputs are projected to 256 dimensions by a learned linear layer (`feat_map`), so text and image tokens share one embedding space, giving `encoded_text` of shape $(B, L, 256)$. We verified the encoder with three tests: the single-caption shape $(1, 4, 256)$, the truncation of a 300-word caption to exactly 256 tokens, and the rejection of empty input.

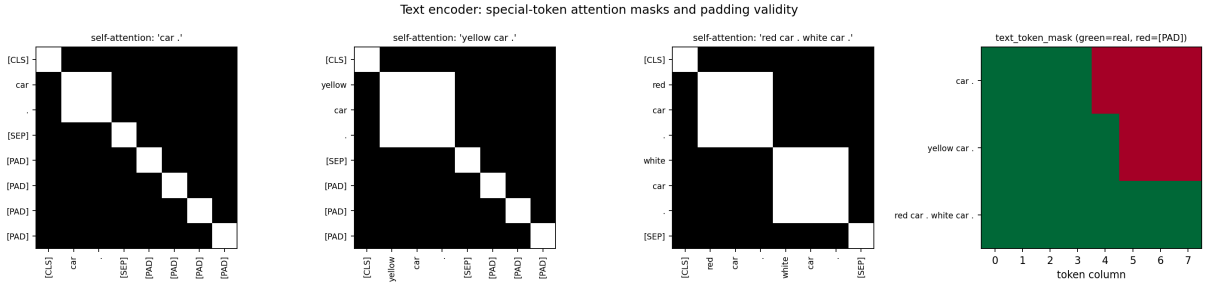


Figure 3: Text encoder masks for three captions of unequal length. Left three panels: special-token self-attention masks (white = may attend); each phrase forms a separate block. Right: validity mask (green = real token, red = [PAD]), showing the padded columns that are excluded from scoring.

3.4 ROIALing Exemplar Extraction

ROIALign is used to extract visual features from a user-defined exemplar region in the input image. The exemplar provides image-specific appearance information about the target object and enables CountGD to perform exemplar-based counting. The extraction process consists of four main stages: defining the exemplar bounding box, transforming its coordinates according to image preprocessing, extracting regional features using ROIALign, and generating an exemplar representation for subsequent model processing.

3.4.1 Exemplar Bounding Box Input

An exemplar bounding box identifies a representative instance of the target object within the input image. The bounding box is specified in xyxy format as $[x1, y1, x2, y2]$, where $(x1, y1)$ represents the top-left corner and $(x2, y2)$ represents the bottom-right corner of the bounding box.

For the car experiment, the exemplar was defined by the bounding box $[55, 72, 151, 218]$ in the original image. This selected region provides visual information about the target object’s appearance, which can subsequently be used to identify and count similar instances within the scene.

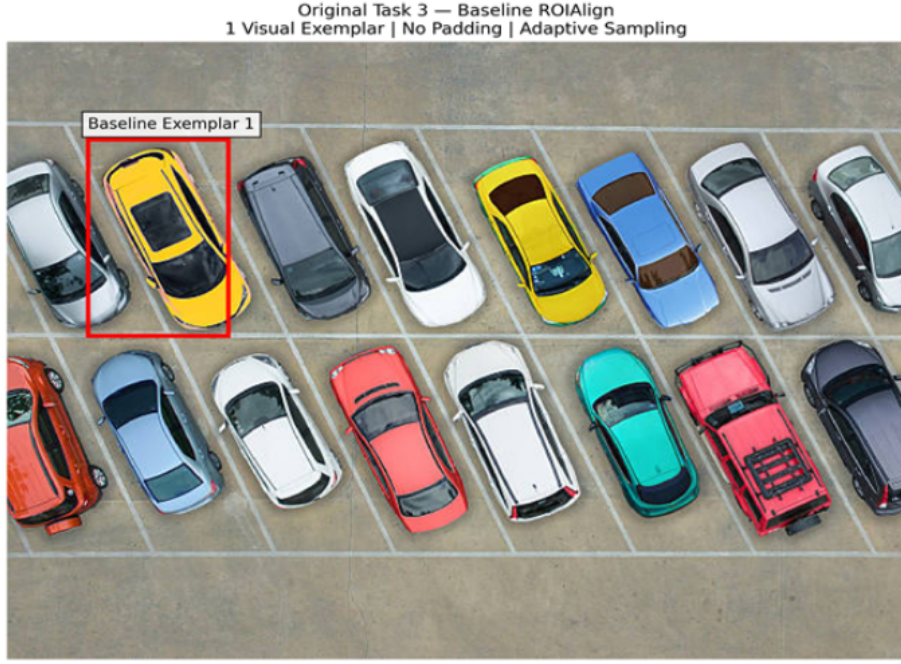


Figure 4: Selected visual exemplar and bounding box for the car image.

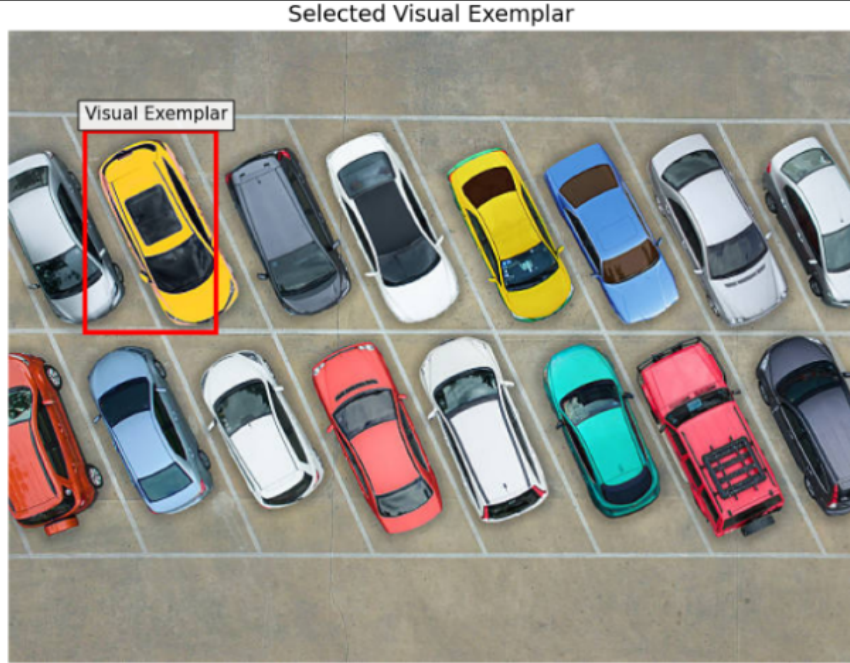


Figure 5: Transformation of the exemplar bounding box from original-image to resized-image coordinates.

3.4.2 Bounding Box Coordinate Transformation

The input image is resized during preprocessing before being passed to the image encoder. Therefore, the original exemplar coordinates must be transformed using the same resizing operation to maintain their alignment with the selected object. The preprocessing pipeline resizes the image to a target short side of 800 pixels, subject to a maximum side length of 1333 pixels.

For the car image, the original bounding box [55, 72, 151, 218] was transformed to approximately [95.80, 125.49, 263.02, 379.96] in the resized image coordinate system. Applying this transformation ensures that

the exemplar region continues to correspond to the same object after preprocessing.

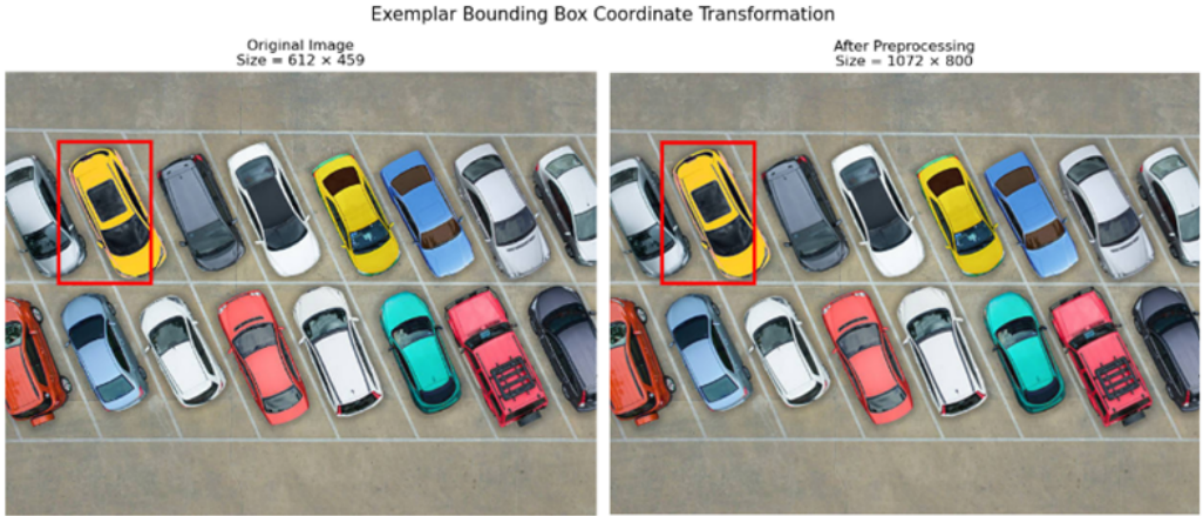


Figure 6: ROIAlign exemplar feature extraction pipeline

3.4.3 ROIAlign Feature Extraction

The transformed exemplar box is used to extract object-specific features from the combined image feature map. For the transformed car image, the image encoder produces a combined feature tensor of shape $(1, 256, 100, 134)$, representing 256 feature channels at approximately one-eighth of the spatial resolution of the resized input image.

ROIAlign is configured using `output_size = (1,1)`, `spatial_scale = 1/8`, and `aligned=True`. The `output_size = (1,1)` setting pools the entire exemplar region into a single spatial feature, enabling one compact representation to be generated for each exemplar. The `spatial_scale = 1/8` setting accounts for the resolution difference between the resized image and the combined feature map, allowing bounding boxes expressed in resized-image pixel coordinates to be correctly mapped to feature-map coordinates. The `aligned=True` setting applies aligned coordinate sampling to reduce spatial misalignment during feature extraction.

For a single exemplar, the ROIAlign operation produces a feature tensor of shape $(1, 256, 1, 1)$. This fixed-size representation captures the visual characteristics of the selected object while remaining independent of the original dimensions of the exemplar bounding box.

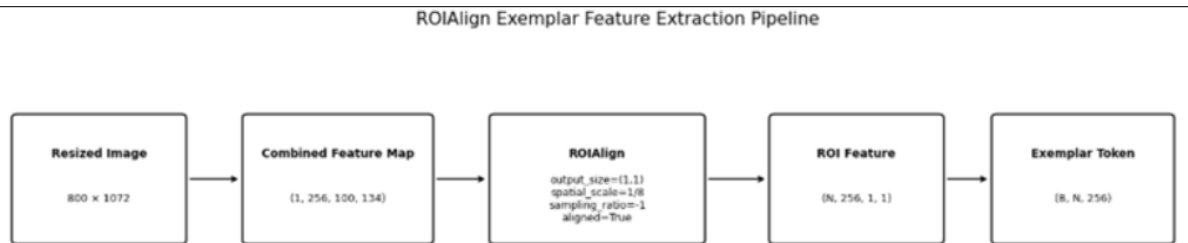


Figure 7: Generation of the 256-dimensional exemplar token using ROIAlign.

3.4.4 Exemplar Token Generation

The ROIAlign output is subsequently converted into the token format required by CountGD. After removing the singleton spatial dimensions and arranging the feature dimensions appropriately, the resulting exemplar token has a shape of $(1, 1, 256)$, corresponding to the model representation (B, N, D) . Here, B denotes the batch size, N represents the number of exemplars, and D is the 256-dimensional feature embedding.

The resulting token provides a compact visual representation of the selected exemplar. This representation enables CountGD to use visual information independently in exemplar-only inference or jointly with a text query in multimodal inference. The report evaluates these prompting configurations under the same model architecture.

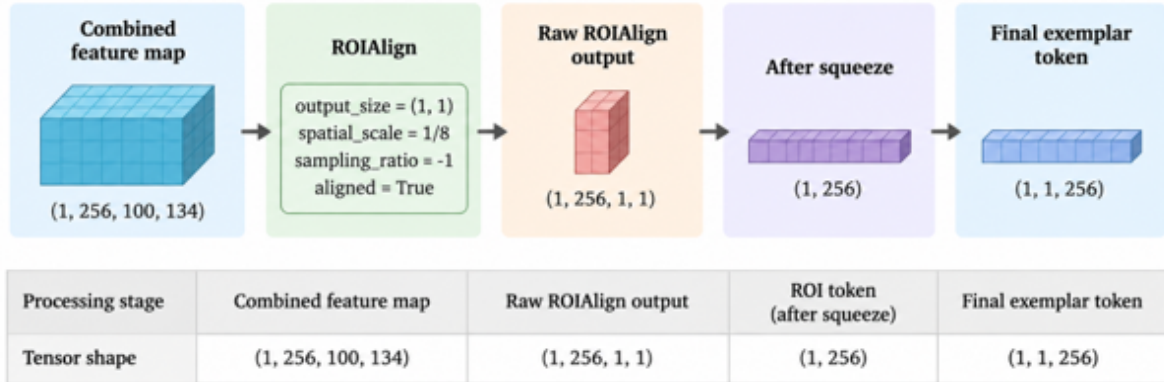


Figure 8: Extraction of the exemplar region from the combined feature map and generation of the $(1, 1, 256)$ exemplar token.

3.4.5 Bounding Box Coordinate Validation

Correct specification of the bounding-box coordinate system is essential for ROIAlign. In the implemented pipeline, the exemplar box is expressed in resized-image pixel coordinates, while $\text{spatial_scale}=1/8$ maps these coordinates to the lower-resolution feature map.

Providing normalized coordinates in the range $[0,1]$ instead would result in incorrect feature extraction. Since $\text{spatial_scale}=1/8$ would also be applied to these normalized values, ROIAlign would sample features from an incorrect region rather than the intended exemplar.

To validate this effect, ROIAlign was applied using both the correct resized-image pixel coordinates and incorrectly normalized coordinates. Both produced a valid $(1, 256)$ feature representation; however, the resulting features had a cosine similarity of 0.694 and an L2 difference of 11.003. This demonstrates that an incorrect coordinate system can substantially alter the extracted exemplar representation even when no runtime error occurs.



Figure 9: Comparison between correct resized-image pixel coordinates and incorrectly supplied normalized coordinates for ROIAlign.

3.5 Similarity Matrix

The function `calculate_similarity_matrix` measures the precision of each image feature matches each prompt token. User queries and prompt features are defined as (B, N, D) and (B, T, D) respectively, in which B is the batch size, N is the number of image positions, T is the number of prompt tokens, and D is the feature size. Then we calculate the whole matrix by multiplying the user queries, and each entry of the matrix is the dot product of one query and one prompt token. The output has the format (B, N, T) . A row describes one image position, and a column will indicate one prompt token. What's more, we set Invalid padding columns to negative infinity so that they cannot win the later maximum operation, to make sure we have the correct answer. For the car example, the grid which has 100×134 features gives 13,400 positions in total, and then four text tokens and one exemplar token give a similarity matrix of $(1, 13400, 5)$. We reshape each column back to the feature grid to draw a heatmap. A brighter area means a higher dot-product score for that token, and thus we can define the distribution of the pixels being marked.

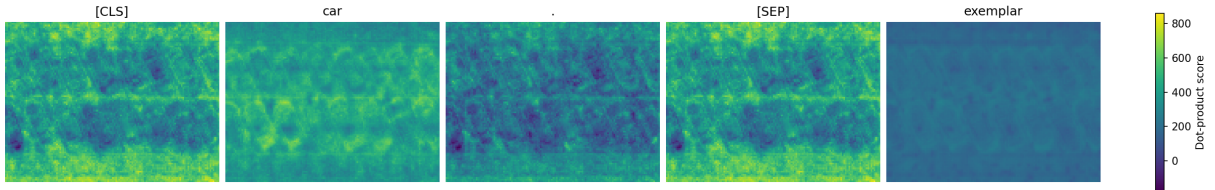


Figure 10: Prompt token heatmaps

3.6 Top-K Proposal Selection

The `select_topk` function in the code first takes the maximum score in each row of the similarity matrix, which also records the ID of the token that gives the corresponding score. Then it uses `torch.topk` function to select the rows that have the highest K scores. Finally, `torch.gather` function will get the last prompt ID for each selected row, and the function will return all the scores, proposal IDs, and prompt IDs data for further analysis. Most importantly, we compare $K = 5, 10, 20$, and 50 using the same features. We also compare four token settings: all valid tokens, no `[CLS]`, no `[CLS]` or `[SEP]`, and only car plus the exemplar. To keep clarity, we separate copies of the similarity matrix so that the original scores will stay unchanged. Top-K selects distinct candidate rows by score, but it does not tell us directly whether they describe the same object. For example, several nearby positions may refer to one car. Therefore, K is not the predicted object count.

Table 1: Top- K overlap and minimum scores under different token-masking settings.

Setting	K	Min score	Overlap	Winning tokens
All tokens	5	789.67	5/5	[CLS]: 5
No CLS	5	786.29	5/5	[SEP]: 5
No CLS/SEP	5	676.12	0/5	car: 5
Car + exemplar	5	676.12	0/5	car: 5
All tokens	10	771.16	10/10	[CLS]: 10
No CLS	10	769.74	10/10	[SEP]: 10
No CLS/SEP	10	664.13	0/10	car: 10
Car + exemplar	10	664.13	0/10	car: 10
All tokens	20	748.69	20/20	[CLS]: 20
No CLS	20	747.39	19/20	[SEP]: 20
No CLS/SEP	20	646.87	0/20	car: 20
Car + exemplar	20	646.87	0/20	car: 20
All tokens	50	722.27	50/50	[CLS]: 50
No CLS	50	720.55	49/50	[SEP]: 50
No CLS/SEP	50	621.85	0/50	car: 50
Car + exemplar	50	621.85	0/50	car: 50

3.7 Prediction and Counting Rules

For each image, after removing the batch dimension, the model returns logits, and boxes have the shape (900, 256) and (900, 4) respectively. Then we apply sigmoid function to the logits and take the maximum token of confidence for each candidate. A candidate is kept only when its score is strictly greater than the threshold, and those predictions with non-finite scores or box values will be removed. A higher threshold means more confidence when models make decisions.

The model gives each box as (cx, cy, w, h), which describes its center, width, and height relative to the image size. We convert these values into pixel coordinates for the box corners and keep them within the image boundaries. After that, we count the boxes that pass the confidence threshold. Two counting rules are used and reported separately. The four baseline tests use the official CountGD rule: each proposal is scored by its highest confidence over all prompt tokens and counted if that score exceeds 0.23 [11]. The changed-prompt experiment additionally applies the phrase rule specified in the project scaffold, which multiplies the confidences of the meaningful words in the prompt and counts proposals whose product exceeds 0.50. The two rules therefore differ in both the scoring operator (maximum versus product) and the threshold. To separate these two effects, we also report the product rule at the official threshold of 0.23.

4. Experiments

4.1 Implementation

Input images were resized to a target short side of 800 pixels with a maximum side of 1333 pixels, converted to tensors, and normalized with ImageNet channel statistics. Each case was timed once after model loading. The measurement included image loading, preprocessing, the forward pass, and post-processing, but excluded checkpoint and BERT initialization.

4.2 Baseline Protocol

Four baseline cases were defined from two source images. The women case used the text-only prompt "women". The car image was evaluated with the text-only prompt "car", exemplar-only input using the xyxy box [55, 72, 151, 218], and a multimodal prompt combining the same box with "car". The preprocessing transform mapped the exemplar to [95.80, 125.49, 263.02, 379.96] in resized pixel coordinates. Each live forward returned 900 queries, with logits of shape (900, 256) and normalized centre-format boxes of shape (900, 4). Post-processing applied sigmoid to the logits, took the maximum token confidence for each query, retained scores strictly greater than $\tau = 0.23$, converted cxcywh boxes to pixel-space xyxy

coordinates, clipped them to the image bounds, and counted the retained predictions. Model weights and image preprocessing were held fixed across cases, so prompt modality was the controlled difference.

4.3 Ablation and Prompt-Sensitivity Protocol

We first compared the three prompt modes on a category word the model cannot ground, “flapper” on `women.jpg`, using one of the model’s own “women” detections as the exemplar. To test how the text query shapes the count, we changed one prompt per image while keeping the image fixed: “women” became “man”, narrowing the category to a subset of the people present, and “car” became “yellow car”, adding an attribute that only a subset of the cars satisfies. We extended this to thirteen probe prompts in three groups: *wording* (“car”, “cars”, “vehicle”, “automobile”; “women”, “woman”, “lady”), *colour* (yellow, red, white and blue car), and *controls* (“parked car”, “shiny car”), whose attribute every car satisfies. True counts for the colour prompts were obtained by manual inspection. All prompt runs use text-only mode with the caption “*query* .”, and every prompt receives its own fresh forward pass.

Two counting rules are reported separately. The *official rule* scores each of the 900 proposals by its maximum confidence over all prompt tokens and counts it if this exceeds 0.23 [11]. The *phrase rule*, specified in the project scaffold and not part of CountGD, multiplies the confidences of the meaningful words only (excluding [CLS], [SEP], punctuation and padding) and counts proposals whose product exceeds 0.50. To separate the effect of the operator from that of the threshold, we also report the product rule at 0.23.

To measure how clearly a prompt separates its targets, we sort a rule’s proposal scores in descending order, $s_1 \geq s_2 \geq \dots$. For a true count n , the predicted count equals n exactly when the threshold t satisfies $s_{n+1} \leq t < s_n$, so we report the *band width* $s_n - s_{n+1}$: a wide band means the targets are ranked first with a clear margin.

5. Results and Discussion

All experiments were run on CPU on a Lenovo Legion 5 (15IRX10) with an Intel Core i9-14900HX (24 cores, 32 threads) and 32 GB RAM, using Python 3.9 and PyTorch 2.2.1 (CPU build) with the official CountGD checkpoint. The machine has an NVIDIA RTX 5070 Laptop GPU, but PyTorch 2.2.1 does not support that GPU generation, so inference ran on CPU at roughly 4 s per forward pass.

5.1 Component Evidence

Table 2 lists the main shapes observed in the notebook. The combined image features and prompt features both use 256 values per token, so their dot products can be calculated. The text test also uses three captions of different lengths to show actual padding. The shorter captions contain [PAD] positions marked as invalid.

The token comparison shows that all selected candidates are led by [CLS] when all valid tokens are included. Removing only [CLS] makes [SEP] the winning token. The candidate sets still share 5/5, 10/10, 19/20 and 49/50 positions with the original sets for $K = 5, 10, 20$ and 50. Removing both special tokens makes car become the winning token, with no overlap with the original sets at these K values. Removing the full stop as well gives the same summary statistics.

Table 2: Main tensor shapes in the car example

Component	Observed shape
Combined image features	(1, 256, 100, 134)
Text features for “car .”	(1, 4, 256)
Visual exemplar token	(1, 1, 256)
Flattened image features	(1, 13400, 256)
Combined prompt features	(1, 5, 256)
Similarity matrix	(1, 13400, 5)
Top- K scores and each ID output	(1, K)
Full-model logits / boxes, no batch axis	(900, 256) / (900, 4)

5.2 Baseline Quantitative Results and Modality Ablation

Table 3 indicates the results for the four prompt cases at thresholds of 0.23. It uses the highest token confidence to select predictions. Figure 11 is the visualisation of the result. As can be seen both numerically and visually, the change in modality did not affect the result of this experiment.

Table 3: Four-case results at a maximum-token threshold of 0.23

Case	Reference	Predicted	Time (s)
Women: text	5	5	3.91
Car: text	16	16	3.94
Car: exemplar	16	16	3.90
Car: text + exemplar	16	16	3.74

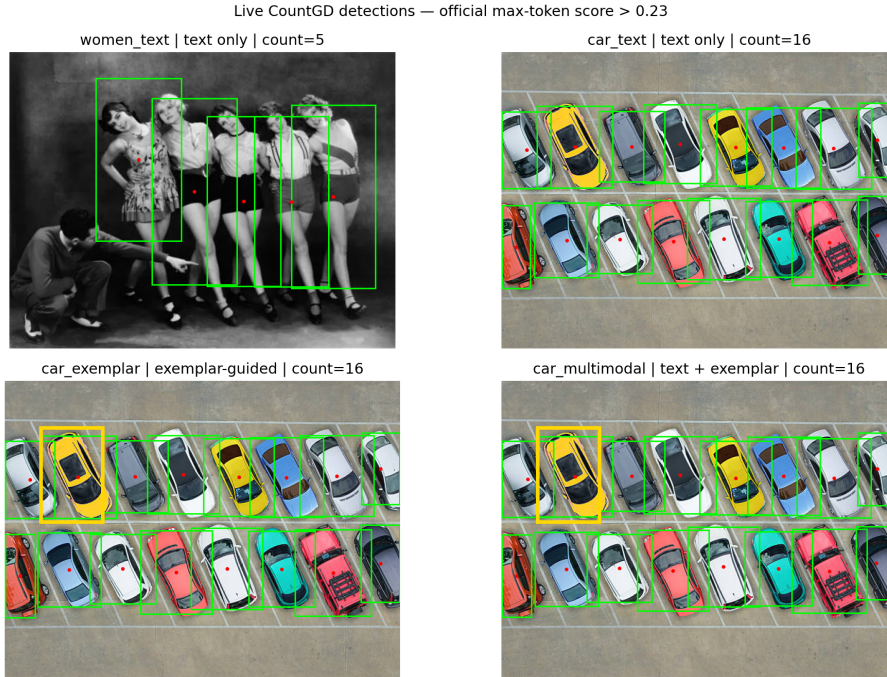


Figure 11: Live CountGD detections - official max-token score > 0.23

The three prompt modes above give identical counts on `car.jpg` (16 in each). We decided to repeat the comparison with a category word the model cannot ground. For “flapper” on `women.jpg`, text-only counting returns no detections as shown in Table 4, although the same subjects are counted correctly with “women” (5). Supplying a visual exemplar, one of the model’s own “women” detections, restores counting: exemplar-only returns 6 detections at a mean confidence of 0.477, and the multimodal prompt returns the same 6 at 0.815 (Table 4). The exemplar recovers counting that text alone cannot express at all, which is the case multimodal design is intended for. It recovers it at the level of *person* rather than *woman*: all six people are counted, including the man, matching what the model returns for the text prompt “person”. Adding the ungrounded word to the exemplar does not change the detections but raises their mean confidence by 0.34.

'flapper' modality ablation — text fails, exemplar recovers counting (true 5)

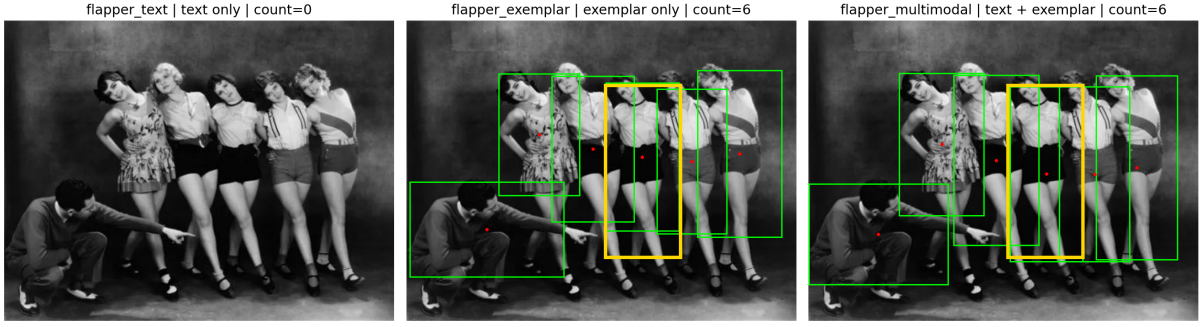


Figure 12: Flapper modality ablation - text fails, exemplar recovers counting

Table 4: Modality ablation on an ungrounded prompt. Counting “flapper” in `women.jpg` (true count 5) under the official rule (maximum over tokens > 0.23); each row is a fresh forward pass. The exemplar is one of the model’s own “women” detections. Text alone finds nothing, while the exemplar restores counting at the level of *person*, including the man; adding the ungrounded word leaves the detections unchanged but raises their mean confidence.

Case	Mode	Caption	True	Count	Mean score
<code>flapper_text</code>	text only	<code>flapper .</code>	5	0	—
<code>flapper_exemplar</code>	exemplar only	<code>object .</code>	5	6	0.477
<code>flapper_multimodal</code>	text + exemplar	<code>flapper .</code>	5	6	0.815

5.3 Prompt Sensitivity and Failure Analysis

5.3.1 Prompt Sensitivity

Under the official rule (maximum over tokens > 0.23), changing the prompt barely changes the count (Table 5): “yellow car” returns all 16 cars instead of 2, and “man” returns 4 instead of 1. Taking the maximum acts as a logical OR, and every car matches “car”. We therefore tested a product of token confidences, intended to act as a logical AND. However, each proposal’s “yellow” and “car” confidences are nearly equal, and even the “car” score peaks on the two yellow cars (Figure 14): the model assigns one confidence to the whole phrase. The product therefore behaves as a stricter threshold on a single score. At 0.23, it still returns 15, and at 0.50, it returns 2 only because the two yellow cars stand clearly above the rest (products 0.558 and 0.555, against 0.426). The maximum rule gives the same result for any threshold between 0.68 and 0.77. For the single token “man”, the product and maximum are identical. Its drop from 4 to 1 is purely a threshold effect (Figure 13). The original prompts are unaffected at 0.50, still returning 5 and 16.

The probe prompts (Table 6) show that category wording is robust: every synonym returns the correct count under both rules with threshold bands 0.60–0.72 wide. However, for every colour, the official rule returns 16, and only yellow is separated with a usable margin. The phrase rule also breaks both controls, returning 15 for “parked car” and 0 for “shiny car”. Since 0.50 was chosen to fix “yellow car”, its success there reflects tuning rather than general improvement.

Table 5: Original and changed prompts under three counting rules. Each row is a fresh forward pass. “Max” is the official CountGD rule; “Product” multiplies the confidences of the meaningful word tokens only. Top score is the highest product score; time is one CPU forward pass.

Image	Prompt	Kind	Tokens	Max > 0.23	Product > 0.23	Product > 0.50	Top score	Time (s)
women.jpg	women	original	women	5	5	5	0.945	3.913
women.jpg	man	changed	man	4	4	1	0.839	4.022
car.jpg	car	original	car	16	16	16	0.978	3.944
car.jpg	yellow car	changed	yellow, car	16	15	2	0.558	3.975



Figure 13: Changed prompts on the same images, one fresh forward pass each. Left: original prompts under the official rule (maximum over tokens > 0.23). Centre: changed prompts under the official rule, where “man” counts the man plus three women (4) and “yellow car” still counts all 16 cars. Right: the same forward passes under the phrase rule (product of word confidences > 0.50), which selects only the man and the two yellow cars.

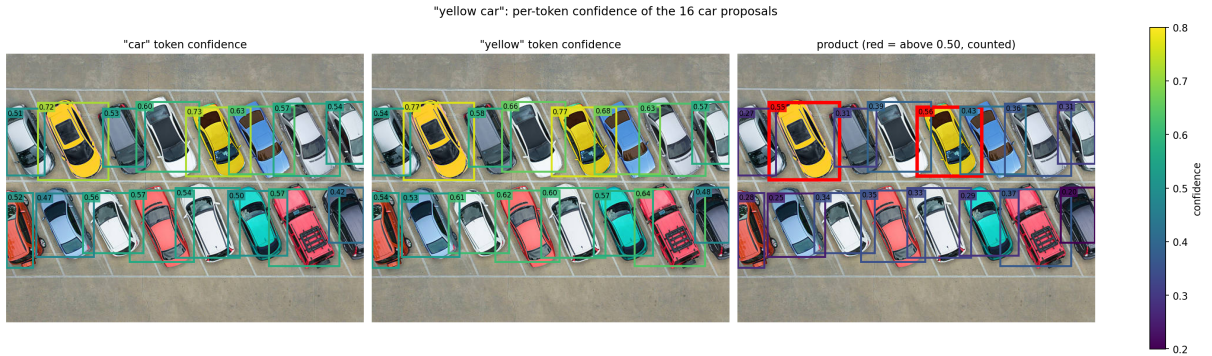


Figure 14: Per-token confidences of the 16 car proposals for the prompt “yellow car”. Left: “car” token; centre: “yellow” token; right: their product, with proposals above 0.50 (counted by the phrase rule) outlined in red. Colour indicates confidence.

Table 6: Prompt probes, one fresh forward pass each. “Band” is the width of the threshold range giving exactly the true count; wider means clearer separation.

Group	Prompt	True	Max > 0.23	Product > 0.50	Max band	Product band
Wording	car / cars / vehicle / automobile	16	16	16	0.60–0.68	0.60–0.68
	women / woman / lady	5	5	5	0.66–0.72	0.66–0.72
Colour	yellow car	2	16	2	0.088	0.128
	red car	3	16	13	0.007	0.009
	white car	3	16	0	0.011	0.018
	blue car	2	16	14	0.001	0.004
Control	parked car	16	16	15	0.646	0.447
	shiny car	16	16	0	0.538	0.299

5.3.2 Failure Analysis

Failure case. For the text-only prompt “red car”, the official rule counts all 16 cars against a true count of 3, and the phrase rule still counts 13, of which only 3 lie on red cars (Figure 15). The threshold range giving the correct count is only 0.007 wide, so no practical threshold recovers it; white and blue cars fail the same way (bands 0.011 and 0.001).

Explanation. As in Section 5.4, the model scores the phrase as a whole, so the “red” token is dominated by “car” and red cars (0.74–0.88) are not separated from the others (0.65–0.86). Unlike yellow, red also spans several ambiguous shades in this image (orange, coral and pink), which makes the attribute harder to ground. This agrees with the original paper, where colour filtering was shown only with exemplars and a separately fine-tuned model [11].

Remedy. We tested two remedies, neither of which recovered the correct count. First, adding one or three red exemplars to the prompt “red car” left the count at 16 and shrank the correct-count band from 0.007 to 0.001, since the exemplars reinforce the *car* category rather than the colour (Figure 15 and Table 7. Second, we removed the category word and followed the form used in the original paper (Figure 16 and Table 16), where the exemplar specifies the object and the text supplies only the attribute: a car exemplar with the text “red” alone still returned all 16 cars, as did the same form with “yellow”, “white” and “blue”. The failure is therefore not an artefact of our prompt format, and an exemplar appears to specify which object category to count rather than which property of it.

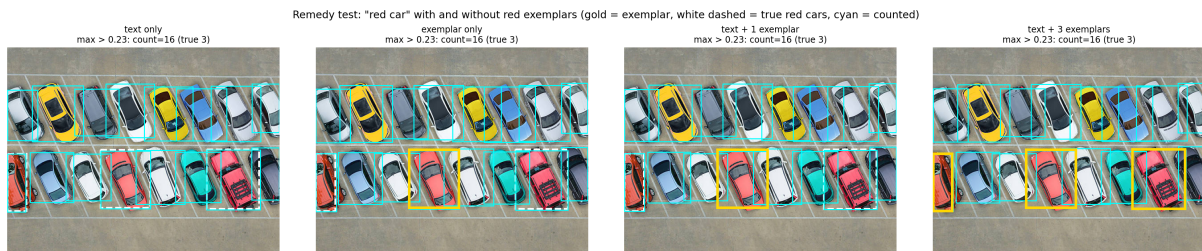


Figure 15: Remedy test for the failure case “red car”. Gold = exemplar, white dash = true red cars, cyan = counted (prediction)

Table 7: Remedy test for the failure case “red car” (true count 3). Each row is a fresh forward pass on `car.jpg`; the exemplar is a box on a red car. “On red” is the number of counted boxes centred on a true red car. “Band” is the width of the threshold range giving exactly 3 under the maximum rule. Time is one CPU forward pass.

Setup	Max > 0.23	On red	Band	Product > 0.50	On red
Text only	16	3	0.007	13	3
Exemplar only	16	3	0.003	—	—
Text + 1 exemplar	16	3	0.001	16	3
Text + 3 exemplars	16	3	0.001	16	3

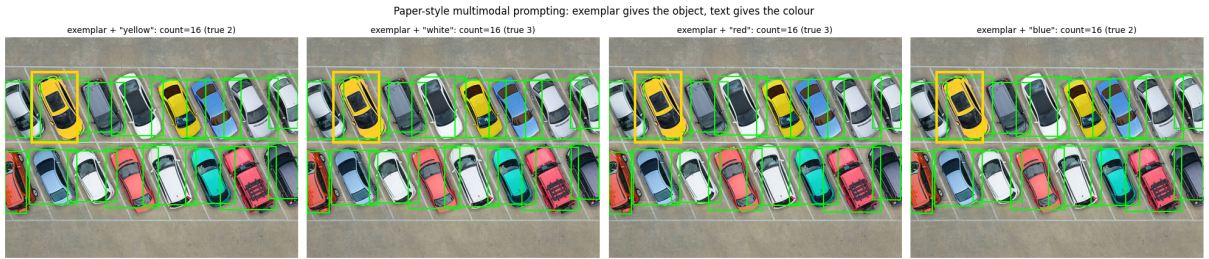


Figure 16: Paper-style multimodal prompting: exemplar gives the object, text gives the colour

Table 8: Paper-style multimodal prompting on `car.jpg`: the exemplar is a box on one car and the text supplies only the colour. Counts under the official rule (maximum over tokens > 0.23); each entry is a fresh forward pass. No setting selects the correctly coloured subset.

Colour	True	Text only	Exemplar only	Exemplar + colour
yellow	2	15	16	16
white	3	16	16	16
red	3	16	16	16
blue	2	16	16	16

6. Conclusion and Limitations

We reproduced CountGD’s inference pipeline component by component and confirmed its tensor shapes and counts with live inference, matching the reference counts on all four baseline cases. The modalities are not interchangeable. For a category word the model cannot ground (“flapper”), text returns no detections while an exemplar restores counting, though at the level of *person* rather than *woman*. CountGD grounds object categories reliably, counting every synonym correctly, but binds colour attributes only weakly. The official rule counts all 16 cars for every colour, and the phrase rule acts only as a stricter threshold.

These findings are illustrative rather than statistical. Each category was tested on a single image, without ground-truth boxes or duplicate suppression, so a correct count does not guarantee one box per object. The colour counts come from our own inspection, and we tested only the released checkpoint, so whether fine-tuning or a per-box colour check would recover colour-specific counts remains open.

7. Team Contributions

Member	Contributions
ALBINOWSKI WIKTOR MARCIN	Image encoder; baselines
OKUMURA MISAKI	Text encoder; sensitivity, modality ablation, failure analysis
MANOHAR RAJA AGATHIYAN	ROIAlign; modality ablation, failure analysis
YAO MUHENG	Similarity/Top-K; validation

Table 9: Contribution of each member

References

- [1] V. Ranjan, U. Sharma, T. Nguyen, and M. Hoai, *Learning To Count Everything*, Apr. 2021. DOI: [10.48550/arXiv.2104.08391](https://doi.org/10.48550/arXiv.2104.08391)
- [2] C. Liu, Y. Zhong, A. Zisserman, and W. Xie, *CounTR: Transformer-based Generalised Visual Counting*, Jun. 2023. DOI: [10.48550/arXiv.2208.13721](https://doi.org/10.48550/arXiv.2208.13721)
- [3] R. Jiang, L. Liu, and C. Chen, “CLIP-Count: Towards Text-Guided Zero-Shot Object Counting,” in *Proceedings of the 31st ACM International Conference on Multimedia*, Oct. 2023, pp. 4535–4545. DOI: [10.1145/3581783.3611789](https://doi.org/10.1145/3581783.3611789)
- [4] A. Radford et al., *Learning Transferable Visual Models From Natural Language Supervision*, Feb. 2021. DOI: [10.48550/arXiv.2103.00020](https://doi.org/10.48550/arXiv.2103.00020)
- [5] N. Amini-Naieni, K. Amini-Naieni, T. Han, and A. Zisserman, *Open-world Text-specified Object Counting*, Sep. 2023. DOI: [10.48550/arXiv.2306.01851](https://doi.org/10.48550/arXiv.2306.01851)
- [6] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, *End-to-End Object Detection with Transformers*, May 2020. DOI: [10.48550/arXiv.2005.12872](https://doi.org/10.48550/arXiv.2005.12872)
- [7] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, *Deformable DETR: Deformable Transformers for End-to-End Object Detection*, Mar. 2021. DOI: [10.48550/arXiv.2010.04159](https://doi.org/10.48550/arXiv.2010.04159)
- [8] S. Liu et al., *Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection*, Jul. 2024. DOI: [10.48550/arXiv.2303.05499](https://doi.org/10.48550/arXiv.2303.05499)
- [9] Z. Liu et al., *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*, Aug. 2021. DOI: [10.48550/arXiv.2103.14030](https://doi.org/10.48550/arXiv.2103.14030)
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, May 2019. DOI: [10.48550/arXiv.1810.04805](https://doi.org/10.48550/arXiv.1810.04805)
- [11] N. Amini-Naieni, T. Han, and A. Zisserman, *CountGD: Multi-Modal Open-World Counting*, Mar. 2025. DOI: [10.48550/arXiv.2407.04619](https://doi.org/10.48550/arXiv.2407.04619)
- [12] K. He, G. Gkioxari, P. Dollár, and R. Girshick, *Mask R-CNN*, Jan. 2018. DOI: [10.48550/arXiv.1703.06870](https://doi.org/10.48550/arXiv.1703.06870)